# STUDYHUB — LEARNING RESOURCES PORTAL

## Unit 4 Project Documentation

Student Name: Ghadah Ammar Alabbadi

Technologies: Python, Django, HTML5 & CSS3

# 1. Project Abstract

StudyHub is a full-stack web application built with Django that lets users browse a catalog of learning resources, view resource details through a tabbed interface, save resources to a personal favorites list, switch between light and dark themes, and submit feedback through a validated form. Unlike a static HTML and CSS site, the project relies on server-side logic to respond differently to each visitor and each request.

The project demonstrates Django's Model-View-Template architecture, dynamic URL routing with path converters, GET query parameters for tab switching, cookie-based theme persistence, session-based favorites without a database, Django Forms with a custom field validator, template inheritance shared across every page, and a custom error page for invalid routes.

# 2. Application Design Documentation

The application is organized around seven pages that all extend a single base template, so the header, navigation bar, and footer stay identical everywhere while each page focuses on one task.

## 2.1 Home Page Design

The home page welcomes the visitor with a short introduction to StudyHub and one call-to-action button leading into the resource catalog. The header and navigation bar shown here come from the shared base template and appear unchanged on every other page.

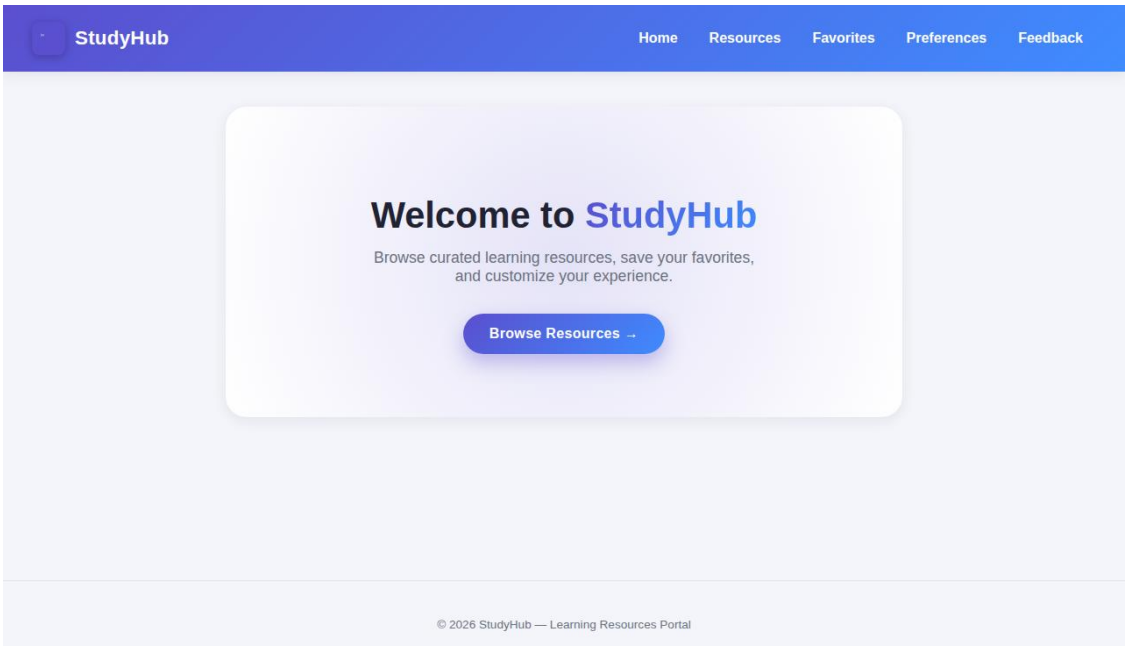



*Figure 1. Home page.*

## 2.2 Resource Catalog

The Resources page lists every learning resource as a card showing its title and a type badge — Article, Video, or Link. Selecting a resource opens its dedicated detail page.

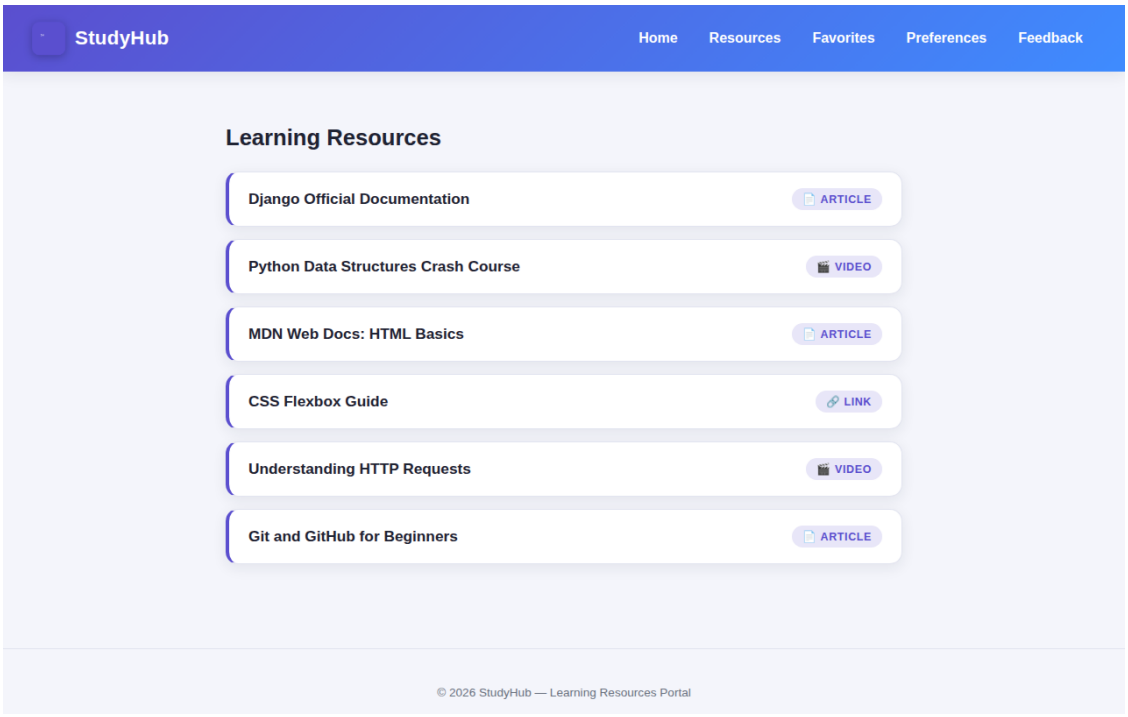



*Figure 2. Resource catalog listing all available learning resources.*

## 2.3 Resource Detail Page and Tabs

Each resource's detail page presents two tabs, Details and Source, switched entirely through a tab value in the URL's query string rather than JavaScript. An Add to Favorites button below the tabs lets the visitor save the resource to their session.

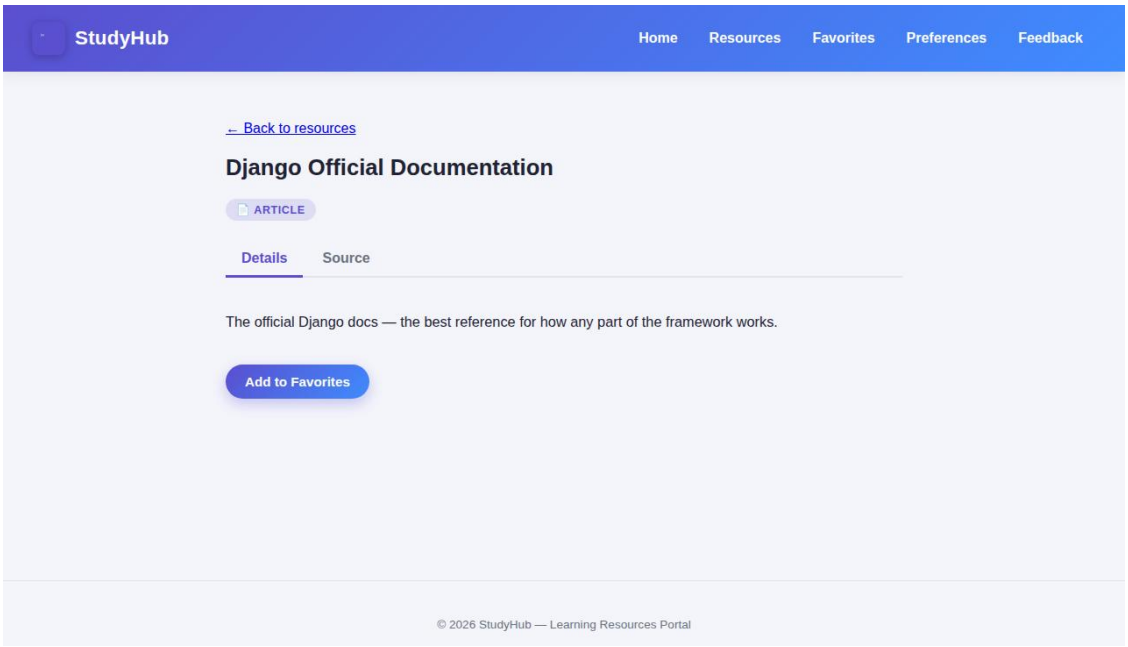



*Figure 3. Resource detail page showing the Details tab.*

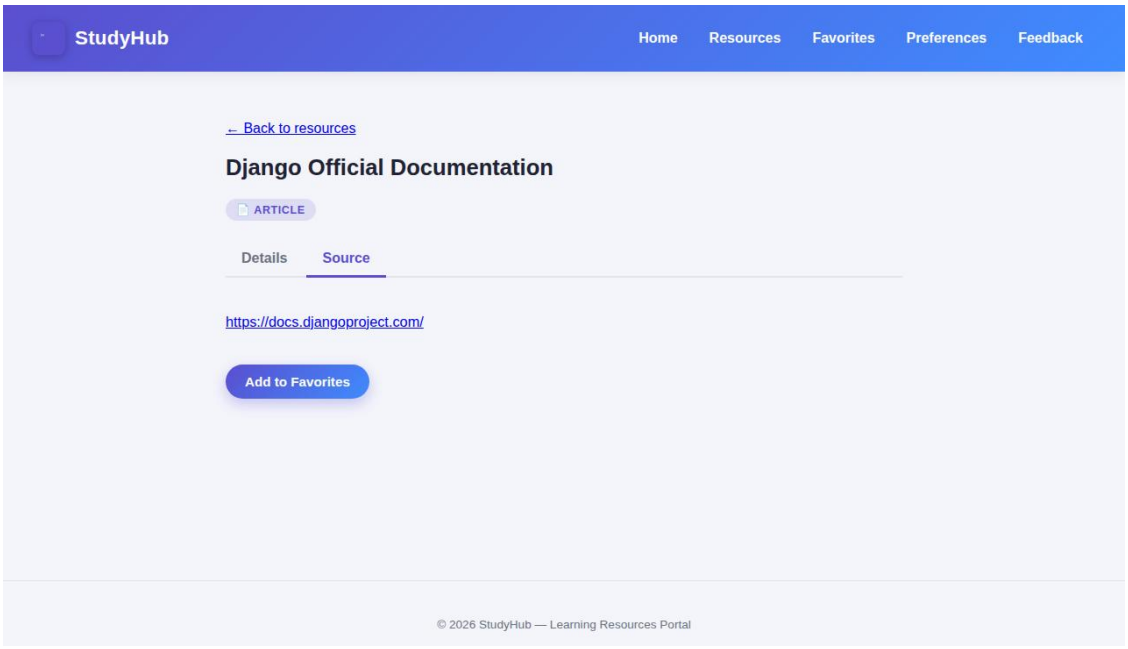



*Figure 4. The Source tab, reached through the ?tab=source query parameter.*

## 2.4 Favorites Page

The Favorites page lists every resource the current visitor has added to their session, each with its own Remove button. Favorites are not written to a database — they live only in the session, so the list clears once the session expires.

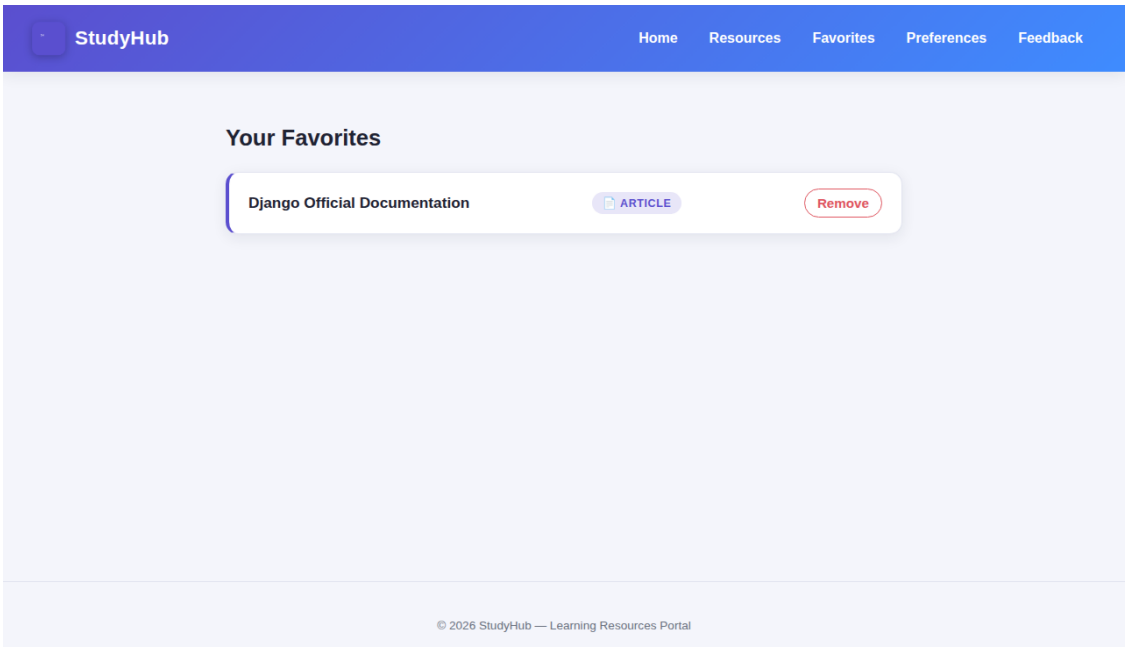



*Figure 5. Favorites page after adding a resource.*

## 2.5 Preferences Page and Theme Switching

The Preferences page lets the visitor choose between a light and a dark theme. The choice is stored in a theme cookie with a one-year expiry, so it persists across visits and is applied consistently on every page, not only on the preferences page itself.

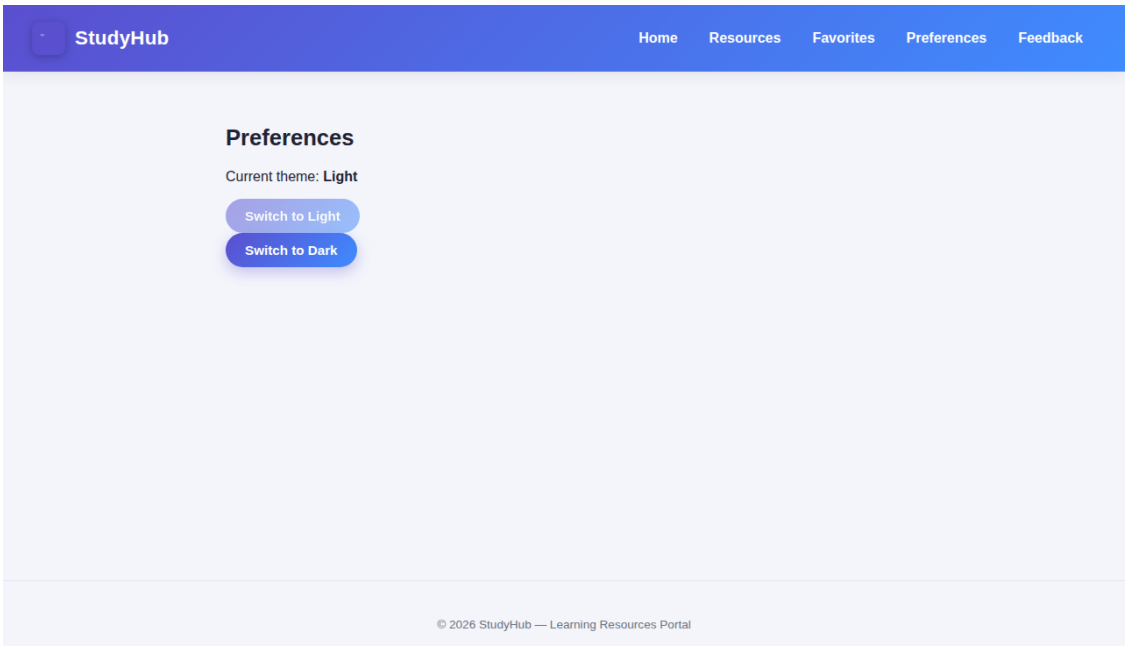



*Figure 6. Preferences page with the light theme active.*

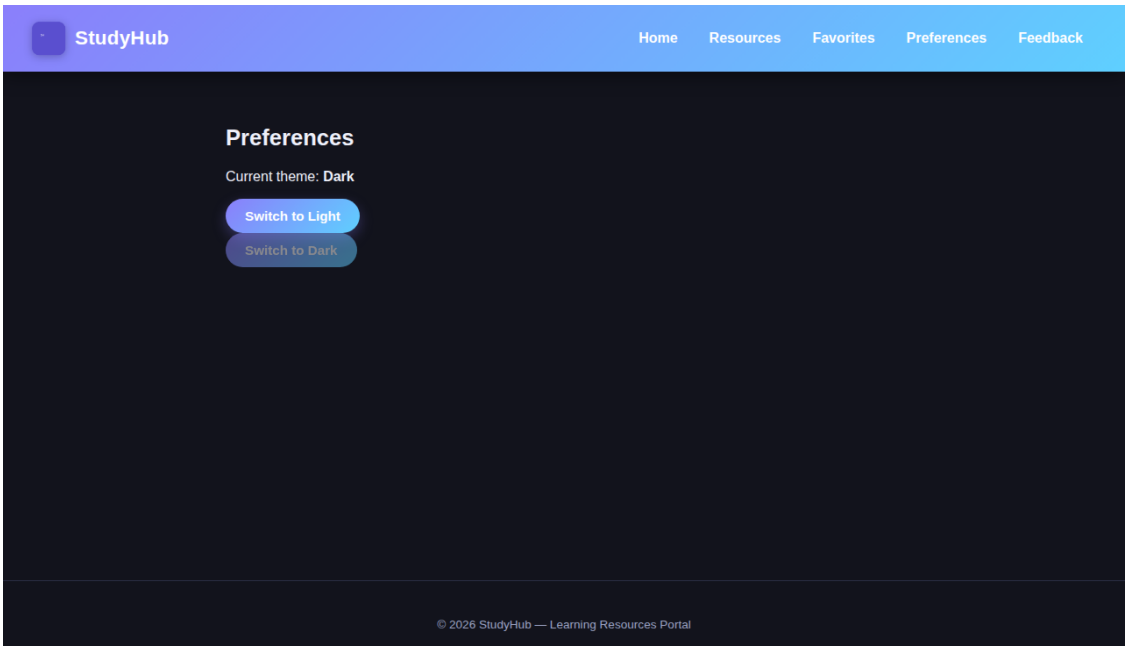



*Figure 7. Preferences page after switching to the dark theme.*

## 2.6 Feedback Form

The Feedback page uses a Django Form with a custom validator requiring the message field to contain at least twenty characters. Each field is rendered manually rather than through the default form.as_p output, so an invalid field can be outlined and its specific error message shown directly beneath it. A valid submission redirects to a confirmation page, following the Post/Redirect/Get pattern so that refreshing the page never resubmits the form.

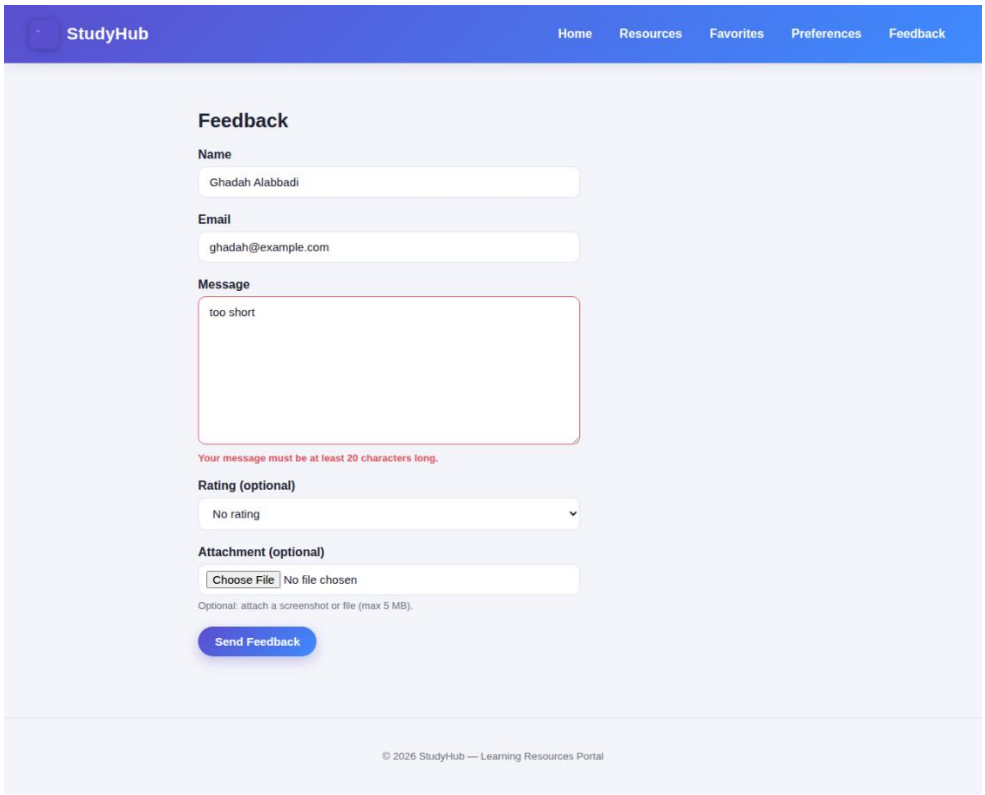



*Figure 8. Feedback form showing a validation error for a message under 20 characters.*

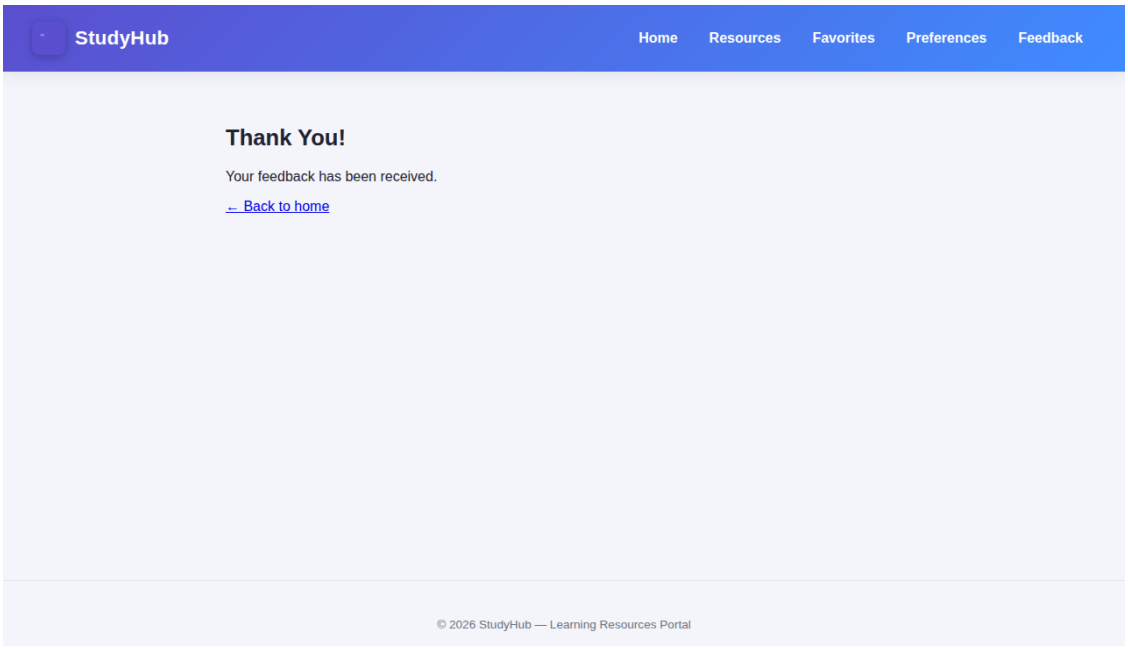



*Figure 9. Confirmation page shown after a valid submission.*

## 2.7 Custom 404 Error Page

Requesting a resource ID that does not exist, or any undefined URL, returns a custom 404 page instead of Django's default debugging page. The page keeps the same header, navigation, and footer as the rest of the site and links back to the resource catalog.

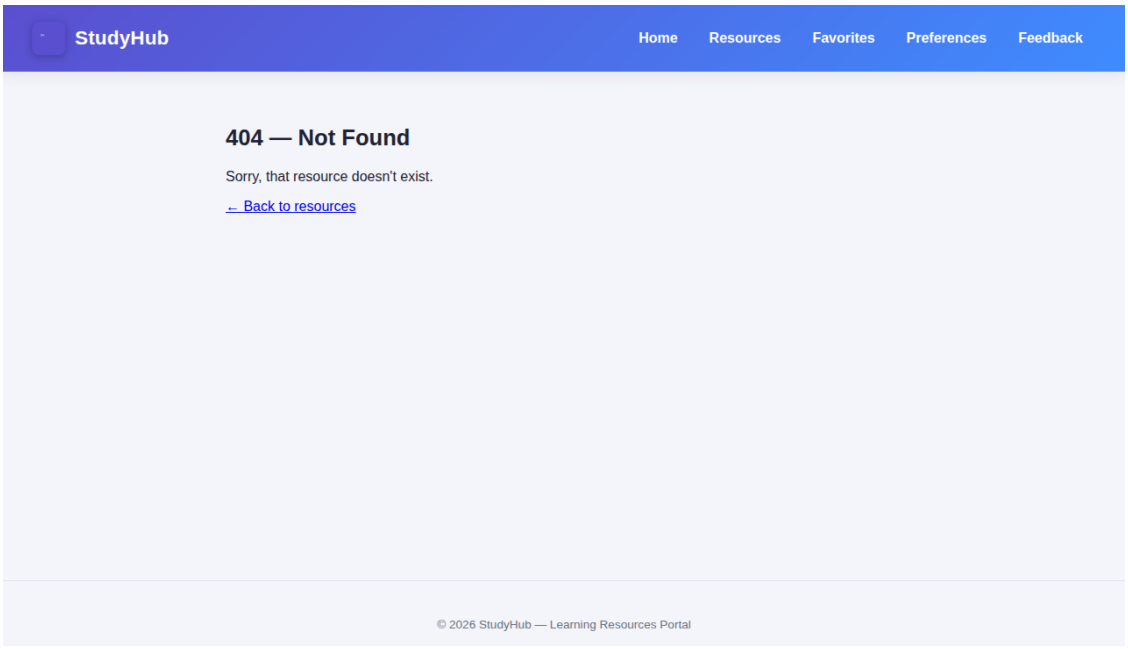



*Figure 10. Custom 404 page for an unknown resource ID.*

# 3. Code Documentation

## 3.1 Project Structure

The project separates the Django project settings, the core application, project-level templates, and static assets into their own folders, following Django's standard layout.

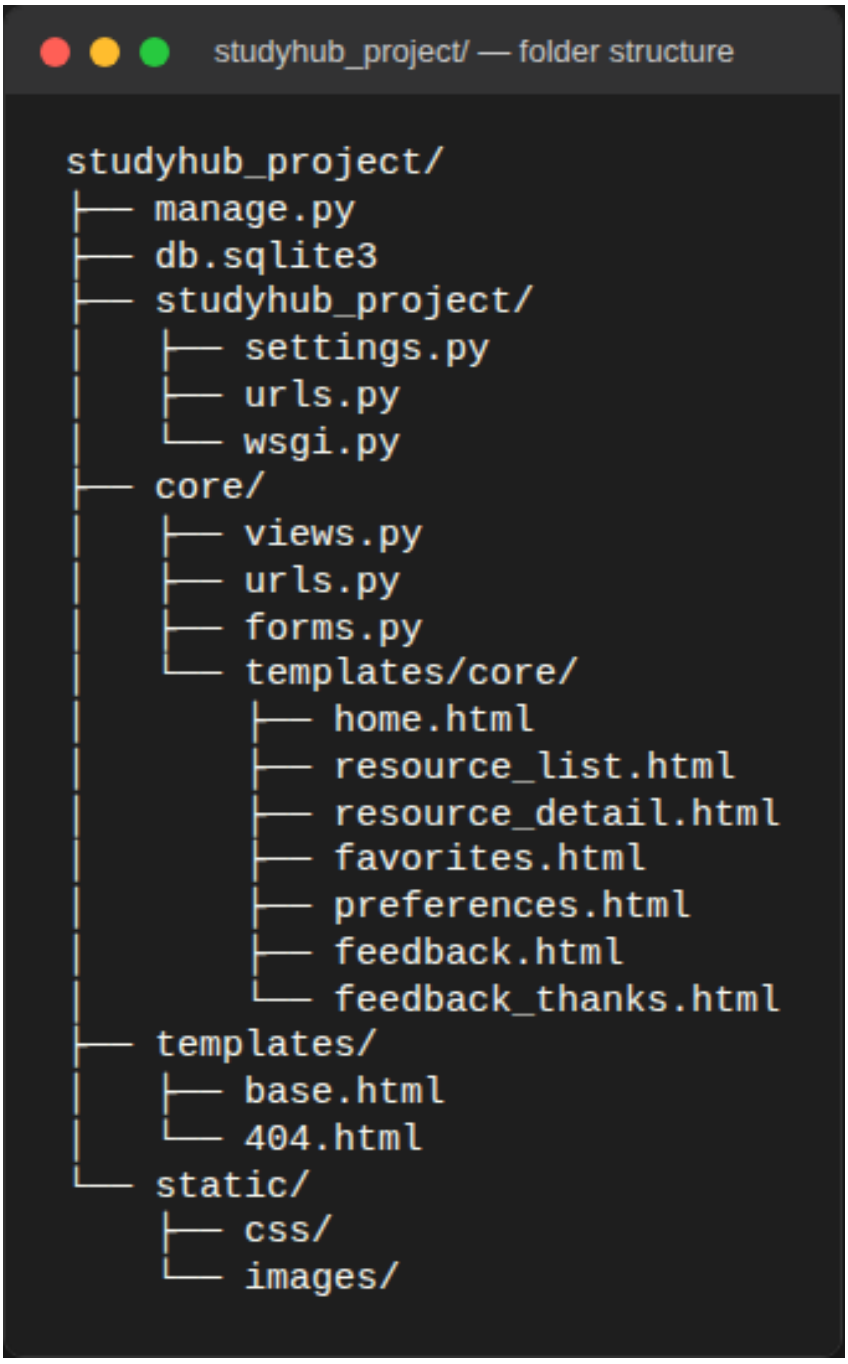



*Figure 11. Project folder structure.*

## 3.2 URL Routing and Views

core/urls.py defines seven named URL patterns under the core namespace, including a dynamic <int:id> path converter for individual resource pages. The project-level urls.py includes this app's URLs with include(), and every template link is generated with {% url %} rather than a hard-coded path.

core/urls.py

```python
from django.urls import path
from . import views

app_name = "core"

urlpatterns = [
    path("", views.home, name="home"),
    path("resources/", views.resource_list, name="resource_list"),
    path("resources/<int:id>/", views.resource_detail, name="resource_detail"),
    path("favorites/", views.favorites_view, name="favorites"),
    path("preferences/", views.preferences, name="preferences"),
    path("feedback/", views.feedback, name="feedback"),
    path("feedback/thanks/", views.feedback_thanks, name="feedback_thanks"),
]
```

*Figure 12. core/urls.py — named URL patterns.*

## 3.3 Query Parameters and Tabs

The resource_detail view reads the tab value from request.GET and falls back to "details" whenever the parameter is missing or holds an unrecognized value, so an invalid or manually edited query string can never break the page. The same view also handles the POST request that adds a resource to favorites.

core/views.py — resource_detail

```python
def resource_detail(request, id):
    resource = get_resource(id)
    if resource is None:
        raise Http404("Resource not found.")

    if request.method == "POST":
        favorites = request.session.get("favorites", [])
        if resource["id"] not in favorites:
            favorites.append(resource["id"])
            request.session["favorites"] = favorites
        return redirect("core:resource_detail", id=id)

    tab = request.GET.get("tab", "details")
    if tab not in VALID_TABS:
        tab = "details"

    favorites = request.session.get("favorites", [])

    return render(request, "core/resource_detail.html", {
        "resource": resource,
        "tab": tab,
        "theme": get_theme(request),
        "is_favorite": resource["id"] in favorites,
    })
```

*Figure 13. resource_detail — reading the tab query parameter and adding a favorite.*

## 3.4 Cookies and Theme Persistence

get_theme() centralizes cookie reading so every view and template receives an already-validated theme value. The preferences view sets the cookie with a one-year max_age only after confirming the submitted value is one of the two supported themes, light or dark.

core/views.py — theme cookie

```python
VALID_THEMES = ("light", "dark")


def get_theme(request):
    """Read the theme cookie, falling back to 'light' if missing or invalid."""
    theme = request.COOKIES.get("theme")
    return theme if theme in VALID_THEMES else "light"


def preferences(request):
    if request.method == "POST":
        theme = request.POST.get("theme")
        if theme not in VALID_THEMES:
            theme = "light"
        response = redirect("core:preferences")
        response.set_cookie("theme", theme, max_age=60 * 60 * 24 * 365)
        return response

    return render(request, "core/preferences.html", {"theme": get_theme(request)})
```

*Figure 14. Reading and writing the theme cookie.*

## 3.5 Sessions and Favorites

Favorites are stored in request.session["favorites"], a plain list of resource IDs. Adding a favorite appends to this list only if it is not already present; removing one deletes it by ID. Because this depends on Django's session framework, the django_session table must exist, which requires running python manage.py migrate once after setup.

core/views.py — favorites_view

```python
def favorites_view(request):
    if request.method == "POST":
        try:
            remove_id = int(request.POST.get("remove_id", ""))
        except ValueError:
            remove_id = None

        if remove_id is not None:
            favorites = request.session.get("favorites", [])
            if remove_id in favorites:
                favorites.remove(remove_id)
                request.session["favorites"] = favorites
        return redirect("core:favorites")

    favorite_ids = request.session.get("favorites", [])
    favorite_resources = [r for r in RESOURCES if r["id"] in favorite_ids]

    return render(request, "core/favorites.html", {
        "resources": favorite_resources,
        "theme": get_theme(request),
    })
```

*Figure 15. favorites_view — removing a favorite from the session.*

## 3.6 Forms and Custom Validation

FeedbackForm is a plain forms.Form rather than a ModelForm, since submissions are not stored in a database. Its clean_message() method uses Django's standard hook for field-level validation to enforce a minimum length beyond what the field type checks on its own.

core/forms.py

```python
from django import forms

RATING_CHOICES = [
    ("", "No rating"),
    ("1", "1 - Poor"),
    ("2", "2 - Fair"),
    ("3", "3 - Good"),
    ("4", "4 - Very good"),
    ("5", "5 - Excellent"),
]


class FeedbackForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
    rating = forms.ChoiceField(choices=RATING_CHOICES, required=False)

    def clean_message(self):
        message = self.cleaned_data["message"]
        if len(message.strip()) < 20:
            raise forms.ValidationError("Your message must be at least 20 characters long.")
        return message
```

*Figure 16. core/forms.py — FeedbackForm with a custom clean_message() validator.*

## 3.7 Templates and Theming

Every page extends templates/base.html, which conditionally loads either light.css or dark.css on top of a shared base.css depending on the current theme. Both theme files define the same set of CSS custom properties with different values, so every component elsewhere in the site only has to reference a variable once to support both themes automatically.

templates/base.html

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/base.css' %}">
{% if theme == 'dark' %}
    <link rel="stylesheet" href="{% static 'css/dark.css' %}">
{% else %}
    <link rel="stylesheet" href="{% static 'css/light.css' %}">
{% endif %}
</head>
<body class="{{ theme|default:'light' }}-theme">
    <header class="site-header">
        ...
    </header>

    <main class="content">
        {% block content %}
        {% endblock %}
    </main>
```

*Figure 17. base.html — conditional theme stylesheet loading.*

static/css/light.css

```css
:root {
    --bg: #f4f5fb;
    --surface: #ffffff;
    --text: #1f2333;
    --text-muted: #6b7280;
    --accent: #5b4fcf;
    --accent-2: #3f8cff;
    --gradient-brand: linear-gradient(135deg, var(--accent), var(--accent-2));
    --danger: #e0555f;
    --border: #e2e4f0;
    --shadow: 0 4px 16px rgba(31, 35, 51, 0.08);
}

body {
    background: var(--bg);
    color: var(--text);
}
```

*Figure 18. light.css — CSS custom properties consumed throughout the site.*

# 4. Conclusion

StudyHub demonstrates the core server-side concepts introduced in Unit 4: dynamic routing, views, templates, forms with custom validation, cookies, and sessions, combined into one consistent Django application. Favorites and feedback are not yet backed by a database, and the project has no user authentication, which leaves clear room to extend it with persistent storage and user accounts in the future.